SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T.Narasimha Reddy	**Reg. No.:**	192425190
Section / Batch:	AIML	**Date:**	20-10-2026

Code of Conduct

I T.Narasimha Reddy / 192425190(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Narasimha Reddy

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1

Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

q0 → q1 → q0 → q1 → q2

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DSA0311 – NATURAL LANGUAGE PROCESSING

ASSESSMENT – 3

Regular Expressions, Finite State Automata, and Pattern Matching

Student Name	T. Narasimha Reddy
Register Number	192425190
Subject	DSA0311 – Natural Language Processing
Assessment	Assessment 3 – Coding Challenge

Introduction

This document presents solutions to the three coding challenges in Assessment 3. The programs focus on input validation using regular expressions, simulation of a deterministic finite automaton (DFA), and a menu-driven pattern matching engine. Each solution includes the problem objective, approach, Python implementation, sample execution, and test cases.

1. Intelligent Email and Password Validator Using Regular Expressions

1.1 Problem Statement

Develop a Python application that validates an email address, password, and mobile number using regular expressions and the rules specified in the assessment.

1.2 Validation Rules

Email: must start with an alphabet; may contain letters, digits, dot, underscore, or hyphen before @; domain must contain only alphabets; extension must be one of .com, .org, .edu, .net, or .in.

Password: minimum 8 characters, with at least one uppercase letter, one lowercase letter, one digit, and one special character from @, #, $, %, &, or !.

Mobile number: exactly 10 digits, with the first digit between 6 and 9.

1.3 Algorithm / Approach

Read the email, password, and mobile number from the user.

Use a regular expression with fullmatch() to check each complete input against its pattern.

Print a separate valid/invalid message for the email and mobile number.

For the password, print Strong Password if all rules match; otherwise print Weak Password.

1.4 Python Program

```
import re

def validate_email(email):
    pattern = r"[A-Za-z][A-Za-z0-9._-]*@[A-Za-z]+\.(com|org|edu|net|in)"
    return re.fullmatch(pattern, email) is not None

def validate_password(password):
    pattern = r"(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}"
    return re.fullmatch(pattern, password) is not None

def validate_mobile(mobile):
    pattern = r"[6-9]\d{9}"
    return re.fullmatch(pattern, mobile) is not None

email = input("Enter email: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")

print("Valid Email" if validate_email(email) else "Invalid Email")
print("Strong Password" if validate_password(password) else "Weak Password")
print("Valid Mobile Number" if validate_mobile(mobile)
      else "Invalid Mobile Number")
```

1.5 Sample Output

```
Enter email: kanna123@gmail.com
Enter password: kanna@123
Enter mobile number: 9876543210
Valid Email
Strong Password
Valid Mobile Number
```

1.6 Test Cases

Input Type	Test Input	Expected Result	Reason
Email	kanna123@gmail.com	Valid Email	Starts with a letter; valid domain and extension.
Email	123kannagmail.com	Invalid Email	Does not start with an alphabet.
Password	kanna@123	Strong Password	Contains upper/lowercase, digit, special character; length ≥ 8.
Password	kanna123	Weak Password	Missing uppercase and special character.
Mobile	9876543210	Valid Mobile Number	10 digits and starts from 6–9.
Mobile	5876543210	Invalid Mobile Number	First digit is outside 6–9.

1.7 Explanation

The email expression checks the required starting character, permitted characters before @, alphabetic domain, and an allowed extension. The password expression uses lookaheads to require each password category. The mobile expression [6-9]\d{9} allows a first digit from 6 to 9 followed by exactly nine digits. fullmatch() ensures that the entire input—not just a substring—matches the rule.

2. Design a Finite State Automata (FSA) Simulator

2.1 Problem Statement

Develop a Python-based Deterministic Finite Automaton (DFA) simulator. It accepts the states, input alphabet, transition table, initial state, final states, and one or more input strings, then displays the transition path and whether each string is Accepted or Rejected.

2.2 Example DFA

Example language: strings over the alphabet {a, b} that end with "ab". The states track the most recent relevant suffix.

Current State	Input a	Input b	Meaning
q0	q1	q0	No useful suffix matched
q1	q1	q2	Last character is a
q2	q1	q0	String currently ends with ab; b breaks it

Initial state: q0. Final state: q2. Input alphabet: a, b.

2.3 Algorithm / Approach

Read the states, alphabet, initial state, final states, and transition table.

For each input string, begin at the initial state and record it in the path.

Read each symbol and use the transition table to move to the next state.

If a symbol has no defined transition, reject the string.

After all symbols are processed, accept only if the current state is a final state.

Display the transition path and final result.

2.4 Python Program

```
def simulate_dfa(states, alphabet, transitions, start_state, final_states, text):
    current = start_state
    path = [current]

    for symbol in text:
        if symbol not in alphabet:
            return path, False
        if (current, symbol) not in transitions:
            return path, False

        current = transitions[(current, symbol)]
        path.append(current)
```

```
    return path, current in final_states



states = {"q0", "q1", "q2"}
alphabet = {"a", "b"}
start_state = "q0"
final_states = {"q2"}

# DFA that accepts strings ending with "ab"
transitions = {
    ("q0", "a"): "q1",
    ("q0", "b"): "q0",
    ("q1", "a"): "q1",
    ("q1", "b"): "q2",
    ("q2", "a"): "q1",
    ("q2", "b"): "q0"
}

print("DFA: strings ending with 'ab'")
while True:
    text = input("Enter a string (or 'quit' to stop): ")
    if text.lower() == "quit":
        break

    path, accepted = simulate_dfa(
        states, alphabet, transitions, start_state, final_states, text
    )
    print("Transition Path:", " -> ".join(path))
    print("Accepted" if accepted else "Rejected")
```

2.5 Sample Output

```
DFA: strings ending with 'ab'
Enter a string (or 'quit' to stop): abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter a string (or 'quit' to stop): abaa
Transition Path: q0 -> q1 -> q2 -> q1 -> q1
Rejected
Enter a string (or 'quit' to stop): quit
```

2.6 Test Cases

Input	Expected Result	Reason
ab	Accepted	Ends with ab.
aab	Accepted	Ends with ab.
aba	Rejected	Ends with a, not ab.
bbaab	Accepted	Ends with ab.
abb	Rejected	Ends with bb.
ac	Rejected	c is outside the alphabet.

2.7 Explanation

A DFA has a finite set of states and exactly one next state for each state-symbol pair in a complete transition table. The simulator updates the current state for every input symbol. Acceptance depends on whether the final state belongs to the set of final states. This example uses a fixed DFA to demonstrate the simulation; the assessment's broader interactive requirement to accept a user-entered transition table would need additional input handling.

3. Smart Pattern Matching Engine Using Regular Expressions

3.1 Problem Statement

Develop a Python-based text search engine that supports word, prefix, suffix, date, phone number, hashtag, and @username/mention searches. The program provides a menu so the user can select the required search type.

3.2 Algorithm / Approach

Store or read a text corpus.

Display a menu of search types.

Read the user's selection and search term where required.

Compile the appropriate regular expression, using case-insensitive matching where suitable.

Collect all matching items and display the matches and count.

Repeat until the user selects Exit.

3.3 Python Program

```
import re
TEXT = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
NLP improves language tools.
"""
def search_pattern(pattern, text, flags=0):
    matches = re.findall(pattern, text, flags)
    print("Matches:", matches if matches else "No matches found")
    print("Count:", len(matches))

def main():
    while True:
        print("\\nSMART PATTERN MATCHING ENGINE")
        print("1. Search Date")
```

```
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")
print("8. Exit")

choice = input("Choose an option: ").strip()

if choice == "1":
    # Matches dates in DD/MM/YYYY format
    search_pattern(r"\\b\\d{2}/\\d{2}/\\d{4}\\b", TEXT)
elif choice == "2":
    search_pattern(r"\\b[6-9]\\d{9}\\b", TEXT)
elif choice == "3":
    search_pattern(r"(?<!\\w)#\\w+", TEXT)
elif choice == "4":
    search_pattern(r"(?<!\\w)@\\w+", TEXT)
elif choice == "5":
    prefix = input("Enter prefix: ").strip()
    if not prefix:
        print("Prefix cannot be empty.")
        continue
    pattern = r"\\b" + re.escape(prefix) + r"\\w*"
    search_pattern(pattern, TEXT, re.IGNORECASE)
elif choice == "6":
    suffix = input("Enter suffix: ").strip()
    if not suffix:
        print("Suffix cannot be empty.")
        continue
    pattern = r"\\w*" + re.escape(suffix) + r"\\b"
    search_pattern(pattern, TEXT, re.IGNORECASE)
```

```
        elif choice == "7":
            word = input("Enter word: ").strip()
            if not word:
                print("Word cannot be empty.")
                continue
            pattern = r"\\b" + re.escape(word) + r"\\b"
            search_pattern(pattern, TEXT, re.IGNORECASE)
        elif choice == "8":
            print("Exiting search engine.")
            break
        else:
            print("Invalid choice. Please select 1–8.")


if _name___== "_main_":
    main()
```

3.4 Sample Output

```
SMART PATTERN MATCHING ENGINE
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Choose an option: 3
Matches: ['#NLP']
Count: 1
```

3.5 Test Cases

Search Type	Expected Match	Expected Count
Date	12/09/2026	1
Phone number	9876543210	1
Hashtag	#NLP	1
Mention	@OpenAI	1
Prefix: nat	natural	1
Suffix: processing	processing	1
Word: NLP	#NLP and NLP	2

3.6 Explanation

The engine uses separate regular expressions for each pattern type. Word search uses word boundaries so it does not match the search term as part of a larger word. Prefix and suffix searches use re.escape() to treat the user's term as literal text rather than as regular-expression syntax. re.IGNORECASE makes word, prefix, and suffix searches case-insensitive. The date pattern recognizes the requested DD/MM/YYYY shape but does not verify calendar validity.

4. Overall Testing and Quality Checks

Test valid inputs and invalid inputs, including boundary cases.

Use fullmatch() for validation tasks to avoid accepting partial matches.

Check empty input, unsupported symbols, and malformed strings.

Verify both the result and the transition path for DFA inputs.

For search tasks, verify the exact matches and the reported count.

5. Rubric Alignment

Criterion	Weight	How this document addresses it
Problem Understanding	20%	Each problem is restated with its stated constraints and expected behavior.
Algorithm	25%	Step-by-step approaches are provided for all three programs.

Code Implementation	20%	Python code is organized into functions and uses regular expressions or DFA transitions as appropriate.
Competitive Performance	20%	The examples use direct regex matching and dictionary-based DFA transitions; test inputs are small and efficient.
Test Case Handling	15%	Valid, invalid, and edge-oriented examples are listed for each program.

Conclusion

The assessment demonstrates three core concepts: regular-expression validation, state-transition simulation, and text pattern matching. The included algorithms, programs, sample outputs, and test cases provide a structured basis for implementation and evaluation against the supplied rubric.